

CREANDO APP USUARIO



>>> PASOS

Para seguir integrando este blog, en este apartado veremos el uso de usuarios que nos provee Django. Para el contenido de este proyecto podemos usar User o AbstractUser o AbstractBaseUser.

Para nuestro caso vamos a usar AbstractUser, por lo que vamos a comenzar creando al app usuario para luego seguir con el modelo.

(entorno) C:\Users\Pc\Proyecto Web\blog\apps>django-admin startapp usuario

Ahora dentro de la aplicación usuario vamos a ir al "models.py"

```
1 from django.db import models
2 from django.urls import reverse
3 from django.contrib.auth.models import AbstractUser
4
5 # models
6
7 class Usuario(AbstractUser):
8     imagen = models.ImageField(null=True, blank=True, upload_to='usuario', default='usuario/user-default.png')
9
10     def get_absolute_url(self):
11         return reverse('index')
```

Como puedes ver hacemos las importaciones pertinentes que usaremos. Dentro de estas está AbstractUser.

Al heredar de AbstractUser, el modelo Usuario incluye todos los campos y métodos del modelo de usuario predeterminado de Django, como nombre de usuario, correo electrónico y contraseña.

Además, en el modelo Usuario agregamos un campo adicional llamado "imagen". Este campo permite a los usuarios subir una imagen de perfil.

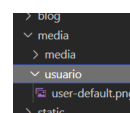
El campo imagen tiene las siguientes propiedades:

- null=True: Esto significa que el campo puede ser nulo en la base de datos.

- blank=True: Esto significa que el campo puede estar en blanco en los formularios.
- upload_to='usuario': Esto especifica el subdirectororio dentro del directorio MEDIA_ROOT donde se guardarán las imágenes subidas.
- default='usuario/user-default.png': Esto especifica la imagen predeterminada que se utilizará si el usuario no sube una imagen.

Finalmente, el modelo Usuario define un método llamado "get_absolute_url()". Este método devuelve la URL absoluta del objeto de usuario. En este caso, el método devuelve la URL correspondiente a la vista con el nombre 'index'. Esto significa que, cuando se llama al método get_absolute_url() en un objeto de usuario, se redirigirá al usuario a la página principal de la aplicación.

Para las imágenes de los usuarios, creamos una nueva carpeta "usuario" dentro de la carpeta "media" donde agregamos una imagen que va a ser la que lleve por defecto un usuario que se registre, si es que no sube una imagen propia, la cual, en tal caso, se guardará dentro de la carpeta "media/usuario".



CONFIGURACIONES DE APP USUARIO



>>> PASOS

Registraremos este modelo en el admin de Django, por lo que abrimos "admin.py" de nuestra app usuario:

```
admin.py
apps > usuario > admin.py
1 from django.contrib import admin
2 from .models import Usuario
3
4 # Register your models here.
5
6 admin.site.register(Usuario)
7
```

Luego en apps.py cambiaremos la ruta de acceso:

```
apps.py
apps > usuario > apps.py > ...
1 from django.apps import AppConfig
2
3 class UsuarioConfig(AppConfig):
4     default_auto_field = 'django.db.models.BigAutoField'
5     name = 'apps.usuario'
6
```

Lo próximo es configurar el settings.py agregando una nueva línea:

```
28 ALLOWED_HOSTS = []
29
30 AUTH_USER_MODEL = 'usuario.Usuario'
31
32 # Application definition
```

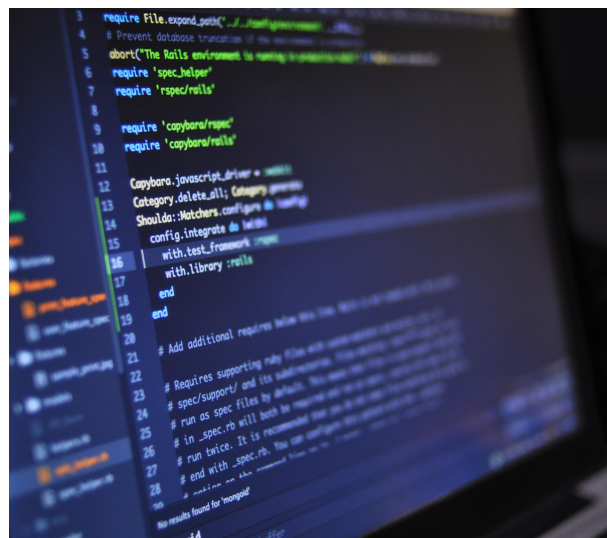
Luego agregamos "apps.usuario" en nuestras aplicaciones:

```
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41
42     'apps.posts',
43     'apps.contacto',
44     'apps.usuario',
45 ]
```

Ahora si quisiéramos hacer las migraciones, nos arrojaría un error ya que hasta este momento estábamos utilizando el usuario "User" de Django, sin embargo, al crear un usuario "AbstractUser", estaríamos reemplazando el usuario actual de Django por lo que se generaría una inconsistencia en la base de datos. Por lo que es momento de enlazar con unos de los temas que los dejamos hasta este momento para que, con el nuevo tipo de usuario, puedas comenzar a usar la base de datos de MySQL.

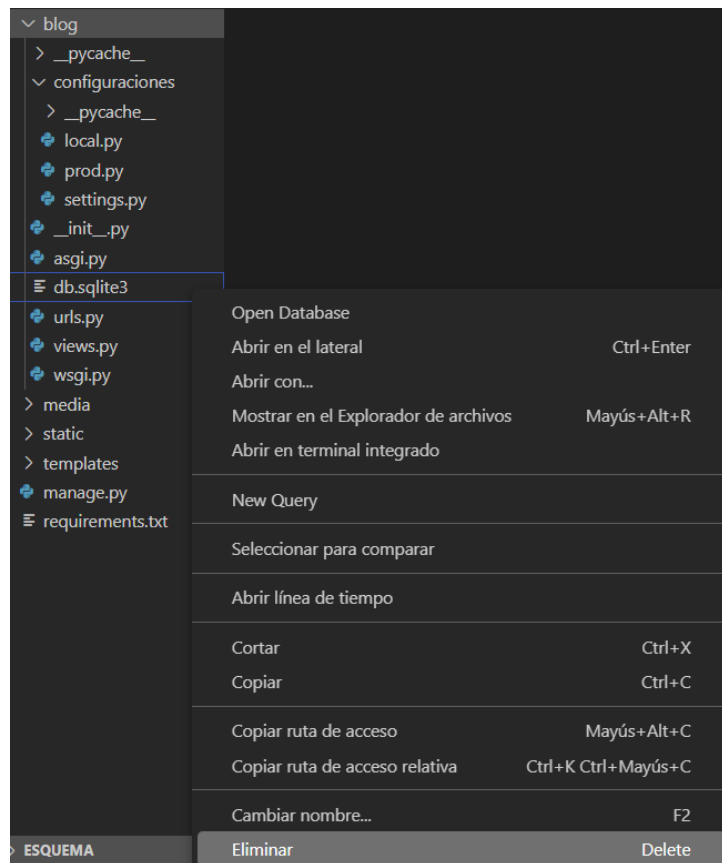
Pero antes de crear esta base de datos, y por si quieres seguir usando sqlite3, vamos a borrar los datos que ya tiene para poder usar AbstractUser.

BORRADO DE SQLITE3



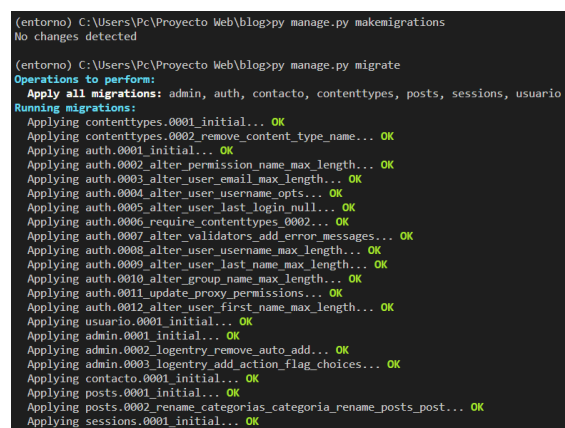
»»» PASOS

Vamos a dirigirnos a la carpeta "blog/blog" y eliminamos nuestra base de datos manualmente con click derecho - Eliminar:



Nos saldrá una ventana donde nos dirá si estamos seguros que queremos eliminar y presionamos en "Mover a la papelera de reciclaje".

Una vez eliminada la base de datos de sqlite3, podemos realizar "makemigrations" y "migrate":



Ahora ya tenemos limpia nuestra base de datos y con "AbstractUser" funcionando en lugar de "User". Para poder volver a ingresar al admin de Django (<http://127.0.0.1:8000/admin/>) debemos crear nuevamente el superusuario como lo hicimos en la página 32. Hecho eso, ya podemos volver a acceder a nuestra base de datos sqlite3, sin embargo, como te mencionamos anteriormente, vamos a comenzar a usar MySQL y para esto te vamos a mostrar 2 formas de crear la base de datos.

MYSQL - CREACIÓN Y ENLACE CON EL PROYECTO



>>> PASOS

Podemos hacerlo desde la consola o desde Workbench. Vamos a hacerlo desde la consola primero para que puedas ver las dos formas.

Primero vamos a abrir una consola cmd, ya sea desde Windows (Inicio - cmd) o desde la terminal en la que estamos trabajando desde VSC y ejecutamos `mysql -u root -p`

Si te llega a salir un error como este:

```
"mysql" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.
```

Debes realizar la siguiente configuración del sistema ya que esto se debe a que el "Path" del sistema no tiene agregado MySQL.

1. Abre el Panel de control y busca la opción "Sistema".
2. Haz clic en "Configuración avanzada del sistema" en el menú de la izquierda.
3. Haz clic en el botón "Variables de entorno".
4. En la sección "Variables del sistema", busca la variable "Path" y haz clic en "Editar".
5. Haz clic en "Nuevo" y agrega la ruta del directorio que contiene las herramientas de línea de comandos de MySQL (por ejemplo, C:\Program Files\MySQL\MySQL Server X.Y\bin).
6. Haz clic en "Aceptar" para guardar los cambios y cerrar todas las ventanas.

Si no hay ningún problema, la consola te pedirá la contraseña configurada.

```
Microsoft Windows [Versión 10.0.19045.3208]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\PC>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.33 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Ahora ya podemos crear la base de datos como ya lo sabés hacer, con el comando `CREATE` seguido del nombre de la base de datos que vamos a crear. En este caso, vamos a llamarla con el mismo nombre del proyecto y agregando las letras db al final y luego para verla vamos a ejecutar el comando `"show databases;"` con el cual se mostrará la base de datos creada. Al final salimos con `"exit"`.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database blogdb;
Query OK, 1 row affected (0.00 sec)

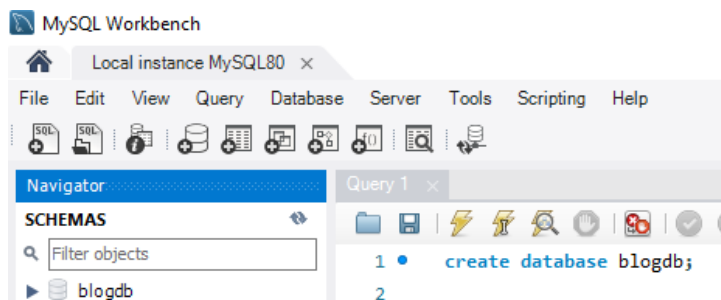
mysql> show databases;
+-----+
| Database |
+-----+
| blogdb   |
+-----+
```


MYSQL - CREACIÓN Y ENLACE CON EL PROYECTO



»»» PASOS

Pero si queremos usar Workbench vamos a abrirlo y ejecutamos el mismo comando para crear nuestra base de datos.

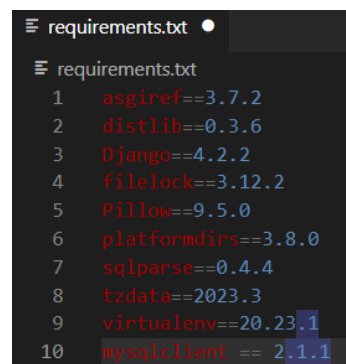


Ahora es momento de modificar el settings.py de nuestro proyecto para comenzar a usar esta base de datos, pero, en nuestro caso, ahora comenzaremos a usar "local.py" de las configuraciones que habíamos hecho al principio. En este archivo declararemos que vamos a usar MySQL. Para ello necesitamos instalar la dependencia de MySQL para Django.

Vamos a la terminal y ejecutamos: **pip install mysqlclient**

```
C:\Users\PC\Proyecto Web\blog>pip install mysqlclient
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: mysqlclient in c:\users\pc\appdata\roaming\python\python311\site-packages (2.1.1)
```

y lo agregamos a requirements.txt. En este caso con la versión que se instaló, pero si usas otra versión, asegurate que sea la misma versión que se instaló al momento de hacer el proyecto.

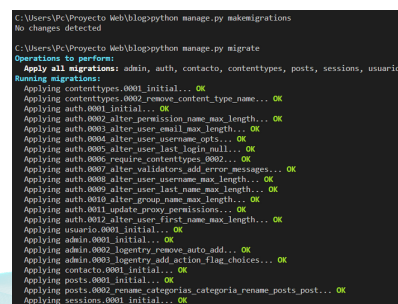


Nos dirigimos a "local.py" y agregamos las siguientes líneas de código poniendo el password que usaremos (en nuestro caso usamos root).

Luego debes dirigirte a "settings.py" y borrar la base de datos declarada ahí:



Guardamos todo y realizamos las migraciones:

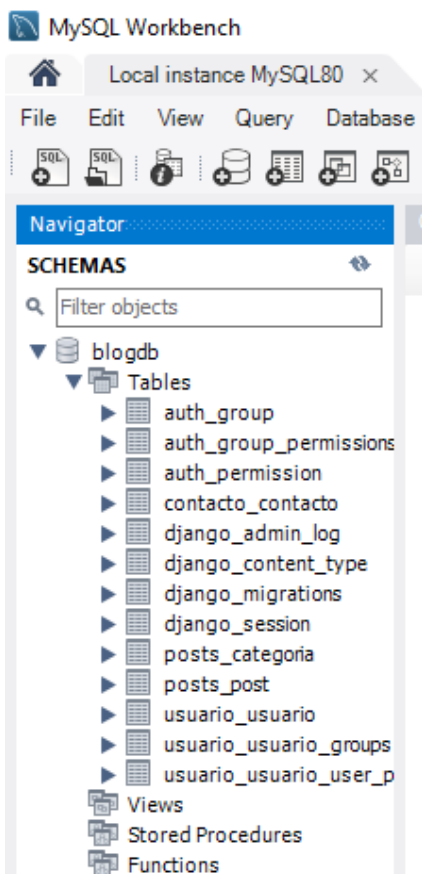


MYSQL - CREACIÓN Y ENLACE CON EL PROYECTO

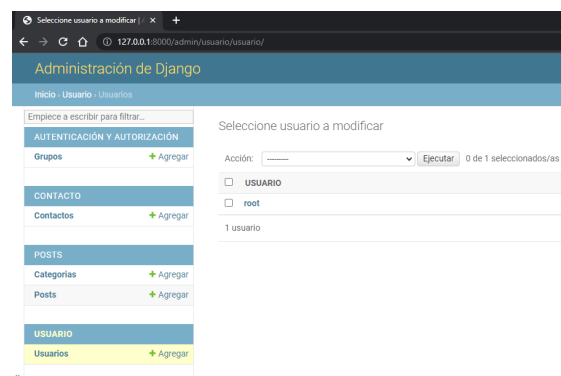


>>> PASOS

Si miramos desde Workbench ahora (o desde la consola cmd), vamos a ver que se crearon correctamente las tablas:



Ahora nuevamente debemos crear el superuser para esta base de datos (página 32) para poder acceder al admin de Django.

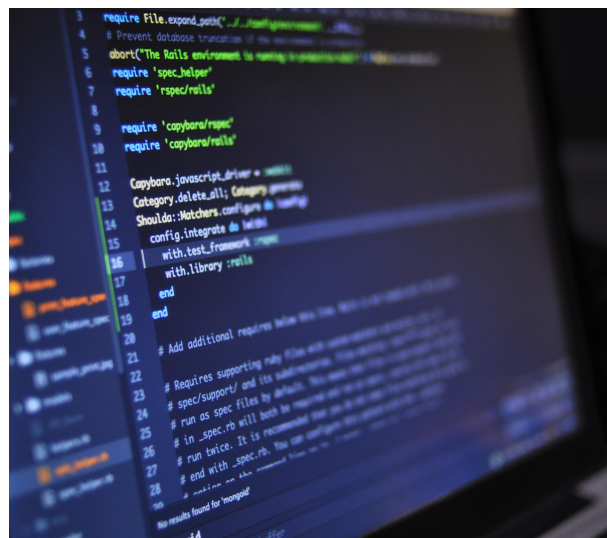


No olvides que teníamos el servidor detenido para hacer estas modificaciones y tienes que volver a correrlo para ingresar al admin nuevamente.

Como verás ahora se separó Usuarios de Grupos y esto es porque ahora estamos usando el AbstractUser para declarar un nuevo usuario en las apps, en vez de User que estuvimos usando hasta el momento.

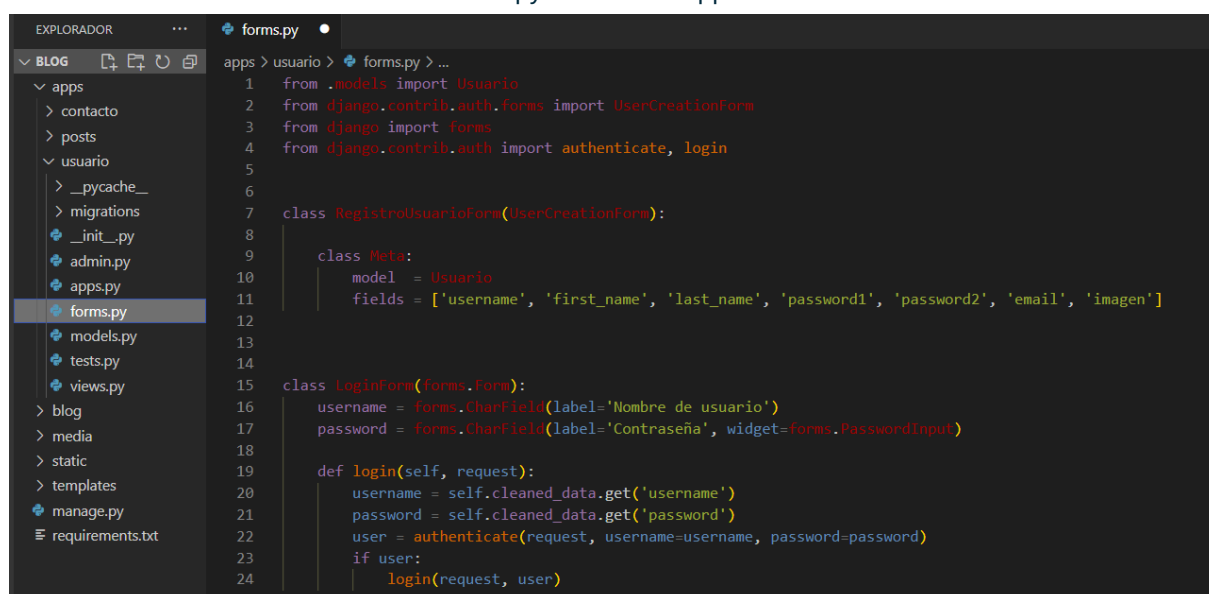
Ahora podemos seguir personalizando nuestro usuario.

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



>>> PASOS

Es momento de crear un login de usuario por lo que necesitamos un formulario para esto y, así que como hicimos con "contacto" vamos a crear un forms.py en nuestra app usuario:



Como puedes ver este código es un ejemplo de cómo se pueden crear formularios en Django para el registro y el inicio de sesión de usuarios.

Primero, se importan las clases necesarias de Django y del modelo Usuario. Luego, se define la clase "RegistroUsuarioForm" que hereda de "UserCreationForm" y se especifica que el modelo a utilizar es "Usuario" y los campos que se van a incluir en el formulario. Después, se define la clase "LoginForm" que hereda de forms.Form y se definen los campos para el nombre de usuario y la contraseña. Finalmente, se define el método "login" que autentica al usuario e inicia sesión si las credenciales son correctas.

El siguiente paso, como lo venimos haciendo, es crear las vistas. En estas vistas basadas en clases que vamos a usar, vamos a estar heredando desde lo que nos provee Django (como habrás leído sobre estas vistas cuando te lo recomendamos en la página 51).

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



>>> PASOS

```
apps > usuario > views.py > ...
1 from .forms import RegistroUsuarioForm
2 from django.contrib.auth.views import LoginView, LogoutView
3 from django.views.generic import CreateView
4 from django.contrib import messages
5 from django.shortcuts import redirect
6 from django.urls import reverse
7
8 # Create your views here.
9
10 class RegistrarUsuario(CreateView):
11     template_name = 'registration/registro.html'
12     form_class = RegistroUsuarioForm
13
14     def form_valid(self, form):
15         messages.success(self.request, 'Registro exitoso. Por favor, inicia sesión.')
16         form.save()
17
18         return redirect('apps.usuario:registro')
19
20 class LoginUsuario(LoginView):
21     template_name = 'registration/login.html'
22
23     def get_success_url(self):
24         messages.success(self.request, 'Login exitoso')
25
26         return reverse('apps.usuario:login')
27
28
29 class LogoutUsuario(LogoutView):
30     template_name = 'registration/logout.html'
31
32     def get_success_url(self):
33         messages.success(self.request, 'Logout exitoso')
34
35         return reverse('apps.usuario:logout')
36
```

Para registrar el usuario heredamos de la vista `CreateView`, como lo hicimos con "contacto" y, de la misma forma también usamos el método para guardar si el formulario es válido con un mensaje incluido. La redirección de la página la dejamos en el registro para poder ver el mensaje de éxito, sin embargo, cuando mejores el código en tu proyecto puedes usar, por ejemplo, `SweetAlert` para mostrar un mensaje emergente del

registro exitoso y que se redirija a la página de iniciar sesión (puedes buscar en la documentación de Django e investigar sobre mensajes: <https://docs.djangoproject.com/en/4.2/ref/contrib/messages/> y sobre mensajes `SweetAlert`: <https://lipis.github.io/bootstrap-sweetalert/>). Lo mismo puedes hacer para "login", "logout" o "contacto" e incluso para "comentarios" (app que crearemos luego).

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS

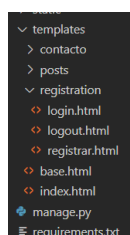


>>> PASOS

Luego para "login" heredamos de la vista que nos proporciona Django "LoginView" donde solo agregamos un mensaje de "Login exitoso" y lo mismo hacemos para "logout". En ambos casos volvemos a quedar en el mismo template para ver los mensajes, pero la idea es que se redirija a la página de inicio, o si estamos en una cierta página, como sería la de un post en particular y queremos hacer un comentario pero debemos loguearnos o registrarnos, debería volver a la misma página del post en el que estamos. Puedes investigar sobre "request.path" (? <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Authentication>).

Esto último se agrega sobre los templates, por lo que en estos enlaces que te dejamos, puedes investigar, sin embargo, ahora nos toca realizar los templates (de forma muy básica) para poder visualizar lo que estuvimos haciendo.

Como pudiste ver en las views.py vamos a usar los templates registrar.html, login.html y logout.html que se encuentran en la carpeta "registration" la cual es una convención de Django ya que estamos usando las vistas que nos provee:



Dentro de estas tres plantillas vamos a usar formularios como lo habíamos hecho con "contacto", ya que, para todos necesitamos trabajar con la base de datos, por lo que llevarán las mismas bases.

Además, como lo hicimos en "contacto" agregaremos código Python para recorrer los mensajes y nos devuelva los mismos.

Te dejamos las vistas http para que veas la forma básica y luego los templates de cada uno.

Iniciar sesión

Login exitoso

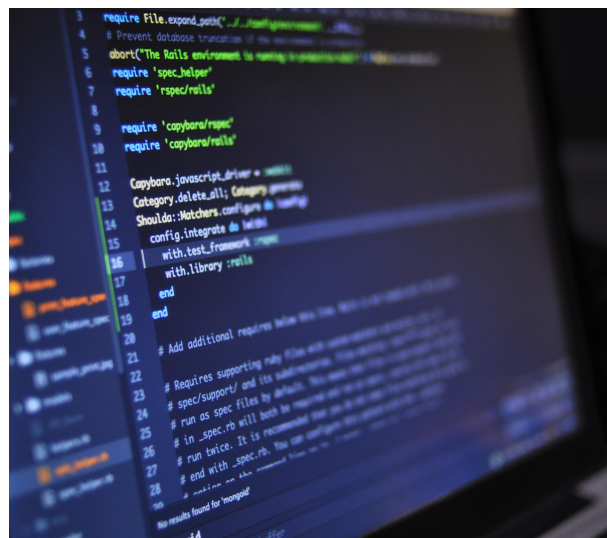
Nombre de usuario:

Contraseña:

Todavía no está registrado? Registrarse

[Iniciar sesión](#)

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



Logout exitoso

Por el momento venimos escribiendo el nombre de la url de forma manual en el http (<http://127.0.0.1:8000/logout/>), pero luego haremos los enlaces y te lo mostraremos. Pero antes te dejamos las plantillas html:

```

templates > registration > registrar.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  <title>{% block title %} Registro de usuario {% endblock title %}</title>
5
6  {% block contenido %}
7  <center>
8
9      <form method="POST" enctype="multipart/form-data" style="margin-top: 300px; margin-bottom: 100px;">
10         <h1 class="h1">Registro de usuarios</h1>
11         <br>
12         <div class="messages">
13             {% for message in messages %}
14                 <table{% if message.tags %} class="{% message.tags %}"{% endif %}>{% message %}</table>
15             {% endfor %}
16         </div>
17         <br>
18         {% csrf_token %}
19         <div class="container-fluid">
20             <table>{% form %}</table>
21         </div>
22         <br>
23         <button type="submit" class="btn btn-success btn-lg px-3">Registrar</button>
24     </form>
25 </center>
26 {% endblock %}

```

```

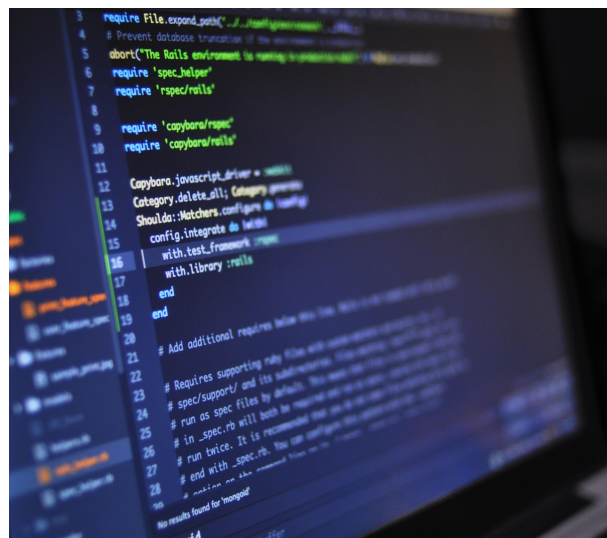
templates > registration > logout.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  {% block contenido %}
5  <center>
6  <div class="container" style="margin-top: 370px;">
7      <div class="messages">
8          {% for message in messages %}
9              <table{% if message.tags %} class="{% message.tags %}"{% endif %}>{% message %}</table>
10          {% endfor %}
11      </div>
12  </div>
13 </center>
14 {% endblock %}

```

Para registrar es muy importante que agregues **enctype="multipart/form-data"** ya que sin este no subirá la imagen cuando la agreguemos en el campo "imagen". El resto del código, como verás es muy similar al de "contacto", por lo que cada vez que queramos un formulario, usaremos básicamente esta estructura. Hay veces que necesitaremos campos específicos y en esos casos utilizaremos "label_tag" (<https://docs.djangoproject.com/en/4.2/ref/forms/api/>).

Logout no llevará demasiado código ya que lo más importante aquí es mostrar el mensaje de logout. Django se ha encargado de realizar el logout correctamente.

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



```
templates > registration > login.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  <title>{% block title %} Registro de usuario {% endblock title %}</title>
5
6  {% block contenido %}
7  <center>
8
9      <form method="POST" style="margin-top: 300px;">
10         <h1 class="h1">Iniciar sesión</h1>
11         <br>
12         <div class="messages">
13             {% for message in messages %}
14                 <table{% if message.tags %} class="{{ message.tags }}" {% endif %}>{{ message }}</table>
15             {% endfor %}
16         </div>
17         <br>
18         {% csrf_token %}
19         <div class="container">
20             <table>{{ form }}</table>
21             <br>
22             <a style="color: black;" href="{% url 'apps.usuario:registrar' %}">Todavía no está registrado? Regístrate</a>
23             <br>
24             <button type="submit" class="btn btn-success btn-lg px-3">Iniciar sesión</button>
25         </div>
26     </form>
27 </center>
28 {% endblock %}
29
30
```

Y para login lo que hicimos fue agregar un botón para que si el usuario no está registrado, pueda acceder directamente al registro de usuario.

Ahora solo nos resta configurar en base.html los accesos desde el Navbar:

```
templates > base.html > html > body > header > nav > div.nav-content > ul.nav-links > li
31 </div>
32 <ul class="nav-links">
33     <li><a href="{% url 'index' %}">Inicio</a></li>
34     <li><a href="#">Acerca de nosotros</a></li>
35     <li><a href="{% url 'apps.posts:posts' %}">Posts</a></li>
36     <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37     <li><a href="{% url 'apps.usuario:login' %}">Login</a></li>
38 </ul>
39 </div>
```

Con esto podremos acceder a loguearnos con nuestro usuario, pero si el usuario no se encuentra registrado, puede acceder desde el mismo template de login al registro de usuarios.

Sin embargo, nos falta una cláusula más para que se habilite un botón de "logout" cuando el usuario está logueado, y eso lo vamos a hacer introduciendo un condicional a nuestro html: (lo vemos en la siguiente página)

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



```

templates > base.html > html > body > header > nav > div.nav-content > ul.nav-links
31 </div>
32 <ul class="nav-links">
33 <li><a href="{% url 'index' %}">Inicio</a></li>
34 <li><a href="#">Acerca de nosotros</a></li>
35 <li><a href="{% url 'apps.posts:posts' %}">Posts</a></li>
36 <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37 {% if user.is_authenticated %}
38 <li><a href="{% url 'apps.usuario:logout' %}">Logout</a></li>
39 {% else %}
40 <li><a href="{% url 'apps.usuario:login' %}">Login</a></li>
41 {% endif %}
42 </ul>
43 </div>
44 </nav>
45

```

Lo que estamos haciendo en este momento, es decirle a Django que verifique que si el usuario está logueado o autenticado, el botón que tiene que aparecer es el de "Logout", pero si no está logueado, tiene que aparecer "Login".

Lo siguiente para finalizar con los usuarios es hacer los accesos de reseto de password por si el usuario se ha olvidado la contraseña.

Esto lo vamos a hacer gracias a que Django por defecto ya tiene instalada la aplicación:

```

33
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41
42     'apps.posts',
43     'apps.contacto'
44 ]

```

Mediante algunas pequeñas configuraciones, vamos a tener el reseteo completo del password del usuario.

Lo primero va a ser agregar en la urls.py principal lo siguiente:

```

blog > urls.py >
18 from django.urls import path, include
19 from .views import index
20 from django.conf import settings
21 from django.conf.urls.static import static
22 from django.contrib.staticfiles.urls import staticfiles_urlpatterns
23
24
25 urlpatterns = [
26     path('admin/', admin.site.urls),
27     path('', index, name='index'),
28     path('', include('apps.posts.urls')),
29     path('', include('apps.contacto.urls')),
30     path('', include('apps.usuario.urls')),
31     path('', include('django.contrib.auth.urls')),
32 ] + static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
33 urlpatterns += staticfiles_urlpatterns()
34 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
35

```

```

31 path('', include('django.contrib.auth.urls'))

```

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



Lo siguiente será crear las urls desde la aplicación usuario:

```
apps > usuario > urls.py > ...
1  from django.urls import path
2  from . import views
3  from .views import LoginUsuario
4  from django.contrib.auth import views as auth_views
5
6  app_name = 'apps.usuario'
7
8  urlpatterns = [
9      path('registrar/', views.RegistrarUsuario.as_view(), name='registrar'),
10     path('login/', LoginUsuario.as_view(), name='login'),
11     path('logout/', views.LogoutUsuario.as_view(), name='logout'),
12     path('password_reset/', auth_views.PasswordResetView.as_view(), name='password_reset'),
13     path('password_reset/done/', auth_views.PasswordResetDoneView.as_view(), name='password_reset_done'),
14     path('reset/<uidb64>/<token>', auth_views.PasswordResetConfirmView.as_view(), name='password_reset_confirm'),
15     path('reset/done/', auth_views.PasswordResetCompleteView.as_view(), name='password_reset_complete'),
16 ]
```

donde tendremos que agregar los "path" para "password_reset", "password_reset/done", "reset/<uidb64>/<token>" y "reset/done". Posterior a esto debemos crear las plantillas para mostrar los formularios y mensajes correspondientes:

- [registration/password_reset_form.html](#): Esta plantilla se utiliza para mostrar el formulario donde los usuarios pueden ingresar su dirección de correo electrónico para solicitar un restablecimiento de contraseña.
- [registration/password_reset_done.html](#): Esta plantilla se utiliza para mostrar un mensaje indicando que se ha enviado un correo electrónico con instrucciones para restablecer la contraseña.
- [registration/password_reset_email.html](#): Esta plantilla se utiliza para generar el correo electrónico que se envía al usuario con instrucciones para restablecer su contraseña. Debe incluir un enlace a la URL `password_reset_confirm` con los argumentos `uidb64` y `token` proporcionados por la vista.
- [registration/password_reset_confirm.html](#): Esta plantilla se utiliza para mostrar el formulario donde los usuarios pueden ingresar una nueva contraseña.
- [registration/password_reset_complete.html](#): Esta plantilla se utiliza para mostrar un mensaje indicando que la contraseña se ha restablecido correctamente.

Por último, antes de crear y completar cada template, debemos configurar un correo electrónico

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



para que se realice el envío del formulario de reseteo de la contraseña. Esta configuración requiere un correo que usemos para hacer el envío del formulario. Si vamos a utilizar un correo distinto cuando trabajemos en producción, te recomendamos realizar la configuración en local.py de nuestras configuraciones y en prod.py el correo que utilizaremos cuando este proyecto se encuentre en producción.

Si vas a usar el mismo correo tanto para el entorno local, como para el entorno de producción, puedes poner esta configuración directamente en settings.py. Nosotros vamos a agregarla en settings.py para el ejemplo.

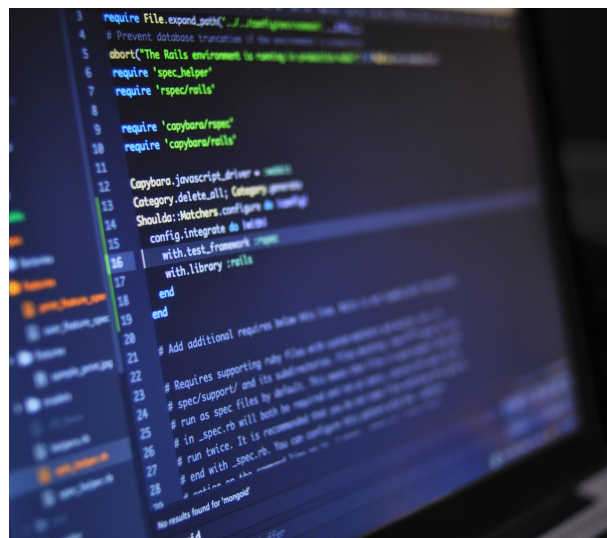
Puedes agregar estas líneas de código en cualquier parte de tu archivo de configuración:

```
blog > configuraciones > settings.py > ...
30 AUTH_USER_MODEL = 'usuario.Usuario'
31
32 EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
33 EMAIL_HOST = 'smtp.gmail.com'
34 EMAIL_PORT = 587
35 EMAIL_USE_TLS = True
36 EMAIL_HOST_USER = 'email_a_utilizar@gmail.com'
37 EMAIL_HOST_PASSWORD = 'contraseña de tu email'
38
39 # Application definition
40
41 INSTALLED_APPS = [
```

Asegurate de reemplazar por tu email y contraseña en esta parte.

Es momento de crear y completar nuestras plantillas para mostrar los formularios y mensajes. Lo vemos en la siguiente página.

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



```
<> password_reset_form.html •
templates > registration > <> password_reset_form.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4      <h2>Restablecer contraseña</h2>
5      <form method="POST">
6          {% csrf_token %}
7          {{ form }}
8          <button type="submit">Enviar correo electrónico</button>
9      </form>
10  {% endblock %}
11  <center>
12      <div class="container" style="margin-top: 370px;">
13          <div class="messages">
14              {% for message in messages %}
15                  <table{% if message.tags %} class="{{ message.tags }}" {% endif %}>{{ message }}</table>
16              {% endfor %}
17          </div>
18      </div>
19  </center>
```

```
<> password_reset_done.html ✕
templates > registration > <> password_reset_done.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4      <center>
5          <h2>Correo electrónico enviado</h2>
6          <p>Se ha enviado un correo electrónico con instrucciones para restablecer tu contraseña.</p>
7      </center>
8  {% endblock %}
```

FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



```
<> password_reset_email.html X
templates > registration > <> password_reset_email.html
1  Hola, has solicitado restablecer tu contraseña. Haz clic en el siguiente enlace para completar el proceso:
2
3  {{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid token=token %}
4
5  Si no has solicitado restablecer tu contraseña, ignora este correo electrónico.
6
7  Saludos,
8  El equipo de {{ site_name }}
```

Aquí no hay que cambiar nada. Django se encargará de todo.

- {{ protocol }}: Esta variable se reemplaza por el protocolo utilizado para generar el enlace de restablecimiento de contraseña (por ejemplo, "http" o "https").
- {{ domain }}: Esta variable se reemplaza por el nombre de dominio de tu sitio (por ejemplo, "www.example.com").
- {% url 'password_reset_confirm' uidb64=uid token=token %}: Esta etiqueta de plantilla genera la URL para la vista de confirmación de restablecimiento de contraseña. Las variables uidb64 y token se reemplazan automáticamente por los valores correctos para el usuario que solicitó el restablecimiento de contraseña.

En cuanto a la variable {{ site_name }}, esta variable se reemplaza por el nombre de tu sitio. Puedes definir el valor de esta variable en la configuración SITE_NAME de tu proyecto de Django. Por ejemplo, si deseas que el nombre de tu sitio sea "Mi sitio", puedes agregar la siguiente línea a tu archivo settings.py:

```
settings.py X
blog > configuraciones > settings.py > ...
38
39  SITE_NAME = 'Informatorio'
40
```


FORMS - VIEW - URL - TEMPLATE PARA USUARIOS



```
<> password_reset_confirm.html X
templates > registration > <> password_reset_confirm.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4    <center>
5      <h2>Restablecer contraseña</h2>
6      <form method="POST">
7        {% csrf_token %}
8        {{ form }}
9        <button type="submit">Restablecer contraseña</button>
10     </form>
11   </center>
12   {% endblock %}
```

```
<> password_reset_complete.html ●
templates > registration > <> password_reset_complete.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4    <center>
5      <h2>Contraseña restablecida</h2>
6      <p>Tu contraseña se ha restablecido correctamente.</p>
7    </center>
8    {% endblock %}
```

Hecho esto, solo queda enlazar las urls correspondiente en el template de login, arriba o abajo de registrarse (lo que comunmente se hace) y con eso ya se completaria la parte básica de usuario.
¡Te lo dejamos para que lo hagas!

COMENTARIOS SOBRE LOS POST



>>> PASOS

Es momento de crear los comentarios para los post. Esto se lo puede hacer desde una aplicación nueva o desde una aplicación existente. Para este caso y simplificarlo más, nosotros vamos a usar la aplicación "post" para agregar los comentarios ahí mismo, por lo que en primera instancia vamos a cargar el modelo en models.py de la app "post". Importamos "settings" que vamos a usar para autenticar a los usuarios:

```
apps > posts > models.py > ...
1  from django.db import models
2  from django.utils import timezone
3  from django.conf import settings
4
```

Creamos el modelo:

```
apps > posts > models.py > ...
35
36  class Comentario(models.Model):
37      posts = models.ForeignKey('posts.Post', on_delete=models.CASCADE, related_name='comentarios')
38      usuario = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='comentarios')
39      texto = models.TextField()
40      fecha = models.DateTimeField(auto_now_add=True)
41
42      def __str__(self):
43          return self.texto
44
```

Donde "posts" es el campo de clave foránea que relaciona cada comentario con un post y si se elimina un post, todos los comentarios relacionados con ese post también se eliminarán debido a la opción `on_delete=models.CASCADE`.

"usuario" es otro campo de clave foránea que relaciona cada comentario con un usuario. Utiliza el modelo de usuario especificado en la configuración `AUTH_USER_MODEL` y establece una relación de "muchos a uno" entre comentarios y usuarios. Cuando se elimina un usuario, todos los comentarios relacionados con ese usuario también se eliminarán debido a la opción `on_delete=models.CASCADE`.

El resto de campo, como el método `__str__` lo basamos en modelos anteriores que creamos.

Lo siguiente que haremos será registrar "Comentario" en el administrador de Django desde admin.py:

```
apps > posts > admin.py > ...
16  admin.site.register(Comentario)
```

COMENTARIOS SOBRE LOS POST



>>> PASOS

Ahora crearemos un nuevo archivo "forms.py" como lo venimos haciendo en las aplicaciones de contacto y usuario, pero ahora lo haremos en la aplicación posts, y lo cargamos:

```
apps > posts > forms.py > ...
1  from django import forms
2  from .models import Comentario
3
4  class ComentarioForm(forms.ModelForm):
5      class Meta:
6          model = Comentario
7          fields = ['texto']
```

Para la vista actualizaremos la vista de "PostDetailView" agregando una función y luego crearemos una nueva vista para comentario:

```
apps > posts > views.py > ComentarioCreateView
14  class PostDetailView(DetailView):
15      model = Post
16      template_name = "posts/post_individual.html"
17      context_object_name = "posts"
18      pk_url_kwarg = "id"
19      queryset = Post.objects.all()
20
21      def get_context_data(self, **kwargs):
22          context = super().get_context_data(**kwargs)
23          context['form'] = ComentarioForm()
24          context['comentarios'] = Comentario.objects.filter(posts_id=self.kwargs['id'])
25          return context
26
27      def post(self, request, *args, **kwargs):
28          form = ComentarioForm(request.POST)
29          if form.is_valid():
30              comentario = form.save(commit=False)
31              comentario.usuario = request.user
32              comentario.posts_id = self.kwargs['id']
33              comentario.save()
34              return redirect('apps.posts:post_individual', id=self.kwargs['id'])
35          else:
36              context = self.get_context_data(**kwargs)
37              context['form'] = form
38              return self.render_to_response(context)
```


COMENTARIOS SOBRE LOS POST



>>> PASOS

Esta función es un método post que se agregamos a la vista que ya teníamos creada para manejar solicitudes POST. Cuando el usuario envíe el formulario de comentario con solicitud POST al servidor con los datos del formulario, este método procesará esa solicitud y realizará las acciones necesarias para guardar los datos del formulario en la base de datos.

```
def post(self, request, *args, **kwargs):
```

>> **Crea una instancia del formulario ComentarioForm con los datos enviados en la solicitud POST**

```
form = ComentarioForm(request.POST)
```

>> **Verifica si el formulario es válido (es decir, si todos los campos requeridos están presentes y son válidos)**

```
if form.is_valid():
```

>> **Si el formulario es válido, guarda los datos del formulario en la base de datos**

```
comentario = form.save(commit=False)
```

```
comentario.usuario = request.user
```

```
comentario.posts_id = self.kwargs['id']
```

```
comentario.save()
```

>> **Redirige al usuario a la vista de detalle del post**

```
return redirect('apps.posts:post_individual', id=self.kwargs['id'])
```

```
else:
```

>> **Si el formulario no es válido, vuelve a mostrar el formulario con los errores de validación**

```
context = self.get_context_data(**kwargs)
```

```
context['form'] = form
```

```
return self.render_to_response(context)
```

Además en el método "get_context_data" agregamos un campo más de contexto para manejar los comentarios que ya están creados y poder mostrarlos:

```
context['comentarios'] = Comentario.objects.filter(posts_id=self.kwargs['id'])
```

Luego creamos la vista de Comentario:

COMENTARIOS SOBRE LOS POST



>>> PASOS

```
apps > posts > views.py > ...
39 class ComentarioCreateView(LoginRequiredMixin, CreateView):
40     model = Comentario
41     form_class = ComentarioForm
42     template_name = 'comentario/agregarComentario.html'
43     success_url = 'comentario/comentarios/'
44
45     def form_valid(self, form):
46         form.instance.usuario = self.request.user
47         form.instance.posts_id = self.kwargs['posts_id']
48         return super().form_valid(form)
```

"ComentarioCreateView" hereda de "LoginRequiredMixin" y "CreateView". "LoginRequiredMixin" es un mixin que requiere que el usuario esté autenticado para acceder a la vista. Si el usuario no está autenticado, será redirigido a la página de inicio de sesión. "CreateView" la hemos visto antes la cual es una vista genérica de Django que proporciona una forma fácil de crear objetos en la base de datos utilizando un formulario.

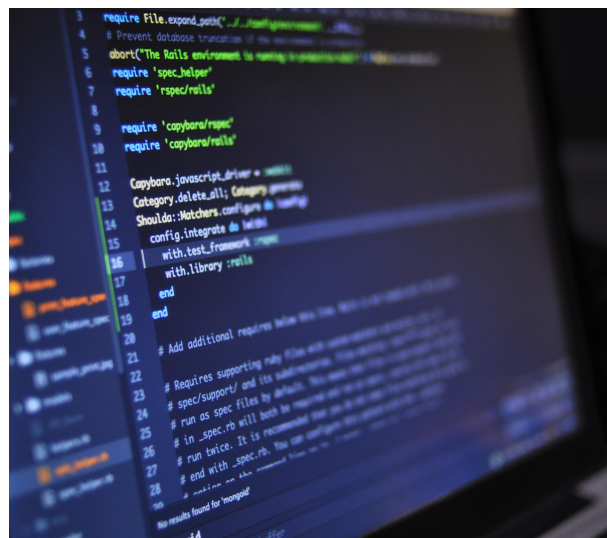
La vista "ComentarioCreateView" está configurada para utilizar el modelo "Comentario" y el formulario "ComentarioForm". Cuando un usuario accede a esta vista, se mostrará un formulario para crear un nuevo comentario utilizando el formulario "ComentarioForm". Cuando el usuario envía el formulario, los datos del formulario se utilizarán para crear un nuevo objeto "Comentario" en la base de datos.

El método `form_valid` se utiliza para establecer el usuario y el post asociados con el comentario antes de guardarlo en la base de datos. El usuario se establece en el usuario autenticado actual (`self.request.user`) y el post se establece en el valor del parámetro "posts_id" pasado en la URL.

Una vez que se ha creado el comentario, el usuario es redirigido a la página actual con el comentario creado. Esto lo vamos a incluir en el template de "post_individual.html"

```
templates > posts > post_individual.html > ...
1 {% extends 'base.html' %}
2 {% load static %}
3
4 {% block contenido %}
5 <center>
6 <div class="container-fluid" style="margin: 200px;">
7
8 <table>{{ posts.titulo }}</table>
9 <table>{{ posts.subtitulo }}</table>
10 <table>{{ posts.categoria }}</table>
11 <br>
12 <table>{{ posts.texto }}</table>
13 
14
15 </div>
16 </center>
```

COMENTARIOS SOBRE LOS POST



>>> PASOS

```
templates > posts > post_individual.html > ...
17 <center>
18 <div class="container-fluid" style="margin-bottom: 20px;">
19   <h4>Comentarios</h4>
20   <br><br>
21 </div>
22 <div class="container-fluid" style="margin-bottom: 20px;">
23   {% for comentario in comentarios %}
24     <table>{{ comentario.usuario }} - {{ comentario.fecha }}</table>
25     <ul class="w-100 list-unstyled d-flex justify-content-between mb-0">
26       <p>{{ comentario.texto }}</p>
27       <br><br>
28     </ul>
29   {% empty %}
30   <li>No hay comentarios - Puedes ser el primero en comentar!</li>
31   {% endfor %}
32 </div>
33 <a id="comentario"></a>
34 <div class="container-fluid" style="margin-bottom: 100px;">
35   <form method="POST" style="margin-bottom: 100px; margin-top: 100px;">
36     {% csrf_token %}
37     {% if user.is_authenticated %}
38     <h2>Deja tu comentario</h2>
39     <form method="POST">
40       {% csrf_token %}
41       {{ form.as_p }}
42       <input type="submit" value="Comentar">
43     </form>
44   {% else %}
45     <h2>Debes iniciar sesión o registrarte para comentar</h2>
46     <a class="btn btn-success btn-lg" href="{% url 'apps.usuario:login' %}?next={{ request.path }}#comentario">Iniciar sesión</a>
47     <input type="hidden" name="next" value="{{ request.path }}">
48   {% endif %}
49 </form>
50 </div>
51 </center>
52 {% endblock %}
```

Por último, antes de poder ejecutar el servidor debemos hacer las migraciones (python manage.py makemigrations y python manage.py migrate) ya que realizamos cambios en el modelo de "post".

Ahora ya tienes las bases para la creación de comentarios. Puedes adaptarlo a tu template para que vaya quedando visiblemente mejor.

MOSTRANDO RESULTADOS



>>> PASOS



Hasta aquí llegamos con esta parte del apunte. Próximamente te presentaremos filtros y algunos ajustes más...